E. Split

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1200 | Tags: binary search, data structures, two pointers | https://codeforces.com/problemset/problem/2148/E

Farmer John has an array a containing n positive integers and an integer k.

Let $a[l, r]$ be a subarray[1] of a. He performs the following procedure to independently determine if subarray $a[l, r]$ is awesome:

- Initially, FJ has k empty multisets, numbered from 1 to k.
- Then, for each element a_i ($1 \le i \le n$) in a:
- If $l \le i \le r$ (that is, a_i is in the subarray $a[l, r]$), he places a_i in multiset 1,
- Otherwise, he places a_i into any multiset he wants **(which may be multiset 1)**.
- Subarray $a[l, r]$ is awesome if there is some way for him to place elements such that, for every value v, all multisets contain the same number of elements with value v. In other words, he wants to make all multisets contain the exact same elements (ignoring ordering).

Output the number of awesome subarrays.

[1]For array a of size n and integers $1 \le l \le r \le n$, the subarray $a[l, r]$ denotes the array consisting of the elements $a_l, \ldots, a_r$, in order.

Input

The first line contains an integer t ($1 \le t \le 1000$) — the number of test cases.

The first line of each test case contains two integers n and k ($2 \le k \le n \le 2 \cdot 10^5$).

The following line contains n space-separated integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each testcase, output one integer on a new line: the number of awesome subarrays.

Example

Input
4 3 2 1 1 1 4 2 1 2 1 2 8 2 3 3 3 3 2 2 2 2 6 3 1 1 1 1 1 1

Output
0 7 18 11

Note

Test case 1: n=3, a=[1,1,1].

For k=2, you cannot finish with the same number of 1's in both multisets, so no subarray works.

Test case 2: n=4, a=[1,2,1,2].

For k=2, the final state must give each multiset exactly one 1 and one 2. Thus a valid subarray can contain at most one 1 and at most one 2.